WORDPRESS ACCESSIBILITY MEETUP — JUNE 2026

Participant Handout

Building Your First Accessible Gutenberg Block

AUDIENCE EDITION — yours to keep. Everything from today, plus extras.

What you'll walk away with

A working block plugin running on your machine.
A complete understanding of edit.js vs save.js — admin view vs visitor view.
WCAG 2.1 AA accessibility built in from line one.
Every command and every line of code, right here in this handout.

00 Requirements — Install Before the Session

You need these two free things

1. Node.js (v18 or higher) → nodejs.org/en/download — choose LTS
2. LocalWP (free local WordPress) → localwp.com

Verify Node.js is installed

```
$ node --version # should show v18.x.x or higher
$ npm --version # should show v9.x.x or higher
```

Windows: if 'node' is not recognised after installing

You need to restart your computer after installing Node.js on Windows.
After restarting, open a NEW Command Prompt window and try again.

01 What Is a Gutenberg Block?

A Gutenberg block is a self-contained, reusable piece of content in the WordPress block editor. It replaces shortcodes with a visual, drag-and-drop component that outputs clean, accessible HTML.

Shortcodes (old way)	Blocks (new way)
[contact-form] [gallery ids='1,2,3']	Visual drag-and-drop in the editor
Memorise syntax. No preview.	Live preview while editing
Breaks if plugin deactivates	Standalone plugin. Works with any theme.

The golden rule — the most important concept

edit.js = what the ADMIN sees in the WordPress dashboard
save.js = what the VISITOR sees on the live website
They can look completely different. That's intentional and powerful.

02 Create Your Block — 5 Steps

Before you start
LocalWP is open, local site is running (green dot)
You can reach wp-admin in your browser
node --version shows v18+ in your terminal

1 Navigate to your plugins folder

In LocalWP → right-click site → 'Go to site folder'. Navigate to wp-content/plugins.

```
# Mac:
$ cd ~/Local\ Sites/my-site/app/public/wp-content/plugins

# Windows:
$ cd C:\Users\YourName\Local Sites\my-site\app\public\wp-content\plugins
```

2 Run the scaffolding command

This one command creates the entire block plugin. Type it exactly:

```
$ npx @wordpress/create-block my-notice-block
# When asked 'OK to proceed?' → type y and press Enter
# Wait ~60 seconds for npm to install dependencies
```

3 Enter the folder and start the compiler

npm start watches your files and recompiles on every save. Leave it running.

```
$ cd my-notice-block
$ npm start
# You should see: webpack compiled successfully
```

4 Activate the plugin

wp-admin → Plugins → find 'My Notice Block' → click Activate.

5 Insert your block

Create a new Page → type / in the editor → search 'notice' → click to insert.

03 File Map — What Does What

File	Purpose — plain English
block.json	Block config: name, title, attributes, supports. WordPress reads this first.
src/edit.js	Admin / editor view — what you see in the dashboard. Import WP tools, export Edit component.
src/save.js	Frontend view — static HTML saved to the database. Use useBlockProps.save().
src/index.js	Entry point — registers the block with WordPress. Rarely needs editing.
src/style.scss	Shared styles — applied in both the editor and on the live website.

04 Complete Code — All Three Files

block.json — attributes section

block.json

```
"attributes": {
"noticeText": {
"type": "string",
"default": "Write your notice here..."
},
"noticeType": {
"type": "string",
"default": "info"
}
},
"supports": {
"color": { "background": true, "text": true },
"spacing": { "padding": true },
"html": false
}
```

src/edit.js — complete file

src/edit.js

```
import { useBlockProps, RichText, InspectorControls }
from '@wordpress/block-editor';
import { PanelBody, SelectControl } from '@wordpress/components';
const ICONS = { info:'■', warning:'■', error:'✕', success:'✓' };

export default function Edit({ attributes, setAttributes }) {
const { noticeText, noticeType } = attributes;
const blockProps = useBlockProps({
className: `notice notice--${noticeType}`,
});
return (
<>
<InspectorControls>
<PanelBody title='Notice settings'>
<SelectControl
```

```
label='Notice type'
value={noticeType}
options={[
{ label:'Info', value:'info' },
{ label:'Warning', value:'warning' },
{ label:'Error', value:'error' },
{ label:'Success', value:'success' },
]}
onChange={(v) => setAttributes({ noticeType: v })}
/>
</PanelBody>
</InspectorControls>
<section {...blockProps}
role="region"
aria-label={`${noticeType} notice`}
>
<span aria-hidden='true'>{ICONS[noticeType]}</span>
<RichText tagName='p' value={noticeText}
onChange={(v) => setAttributes({ noticeText: v })}
placeholder='Write your notice here...' />
</section>
</>
);
}
```

src/save.js — complete file

src/save.js

```
import { useBlockProps, RichText } from '@wordpress/block-editor';
const ICONS = { info:'■', warning:'■', error:'✕', success:'✓' };

export default function save({ attributes }) {
const { noticeText, noticeType } = attributes;
const blockProps = useBlockProps.save({
className: `notice notice--${noticeType}`,
});
return (
<section
{...blockProps}
```

```
role="region"
aria-label={`${noticeType} notice`}
aria-live="polite"
>
<span aria-hidden="true">{ICONS[noticeType]}</span>
<RichText.Content tagName='p' value={noticeText} />
</section>
);
}
```

05 Accessibility — Why Each Line Exists

`role="region"`

Creates a navigable landmark.

Screen reader users jump between landmarks with one keystroke (like skipping chapters). WCAG 1.3.1, 4.1.2.

`aria-label="info notice"`

Gives the landmark a readable name.

role=region without aria-label is ignored. Without it, there's a landmark with no identity. WCAG 4.1.2.

`aria-live="polite"`

Announces dynamic changes without interrupting.

If content changes after load, screen readers announce it when idle. Never use 'assertive' unless it's a genuine emergency. WCAG 4.1.3.

`aria-hidden="true"`

Hides decorative icons from the accessibility tree.

The icon is already described by aria-label. Announcing it again = duplicate. WCAG 1.1.1.

`<section> not <div>`

Semantic HTML — structure that carries meaning.

A = 'generic container'. A <section role=region aria-label=...> = navigable landmark. WCAG 1.3.1.

`:focus-visible in CSS`

Keyboard users always see which element is focused.

NEVER write *:focus { outline: none }. Focus ring must have 3:1 contrast ratio (WCAG 2.4.7, 2.4.11).

06 The Settings Panel + Extending Your Block

Opening the settings panel

→ Insert your block (type / → search 'notice' → click to insert)

→ Click anywhere on the block — a blue outline appears

→ Look at the RIGHT sidebar → you should see 'Notice settings'

→ If hidden: click the ■ icon (top-right of editor toolbar)

Available controls — same pattern for all

Control	What it looks like	Use for
SelectControl	Dropdown menu	Notice type, alignment, size
TextControl	Text input	Custom labels, URLs
ToggleControl	On/off switch	Show/hide elements
RangeControl	Slider	Font size, spacing
ColorPicker	Colour picker	Custom background/text

Add colour + spacing panels for free

block.json supports — free UI panels

```
// In block.json — just add to "supports":
"supports": {
"color": { "background": true, "text": true },
"spacing": { "padding": true }
}
// WordPress adds those panels automatically. No extra JS needed.
```

07 Troubleshooting — 8 Common Errors

'node' is not recognised / command not found

Why: Node.js not installed, or Command Prompt not restarted after install.

Fix: Install from nodejs.org (LTS). Windows: restart computer, open new Command Prompt.

Block doesn't appear in the / inserter

Why: Plugin not Activated, OR npm start isn't running, OR build failed.

Fix: Check 1: wp-admin → Plugins → Activated? Check 2: npm start running?

webpack compiled with errors

Why: Syntax error in a .js file.

Fix: Read the terminal output — it shows the exact file and line. Common: missing comma, unclosed bracket.

'This block contains unexpected or invalid content'

Why: save.js changed after content was already saved in the database.

Fix: Remove the block from the page and re-add it. Use a fresh test page while developing.

Changes to edit.js don't appear

Why: npm start stopped, or browser cached the old version.

Fix: Restart npm start. Do a normal refresh (F5/Cmd+R) — NOT Ctrl+Shift+R.

TypeError: Cannot read properties of undefined

Why: Using an attribute not yet defined in block.json.

Fix: Open block.json → check 'attributes' section. Name must match exactly (case-sensitive).

Block appears in editor but not on frontend

Why: save.js returns null or has an empty return.

Fix: Open save.js — confirm return has full JSX. Use RichText.Content (not RichText).

'npx' is not recognised

Why: npm is not on your PATH.

Fix: Reinstall Node.js. On Mac: brew install node (requires brew.sh).

Resources

Node.js download nodejs.org/en/download
LocalWP localwp.com
VS Code code.visualstudio.com

@wordpress/create-block developer.wordpress.org/block-editor/reference-guides/packages/packages-create-block/
Block Editor Handbook developer.wordpress.org/block-editor/
WordPress Components developer.wordpress.org/block-editor/reference-guides/components/
WCAG 2.1 Quick Reference www.w3.org/WAI/WCAG21/quickref/
axe DevTools (free) deque.com/axe/devtools/
WebAIM Contrast Checker webaim.org/resources/contrastchecker
WP Accessibility Meetup equalizedigital.com/wordpress-accessibility-meetup/